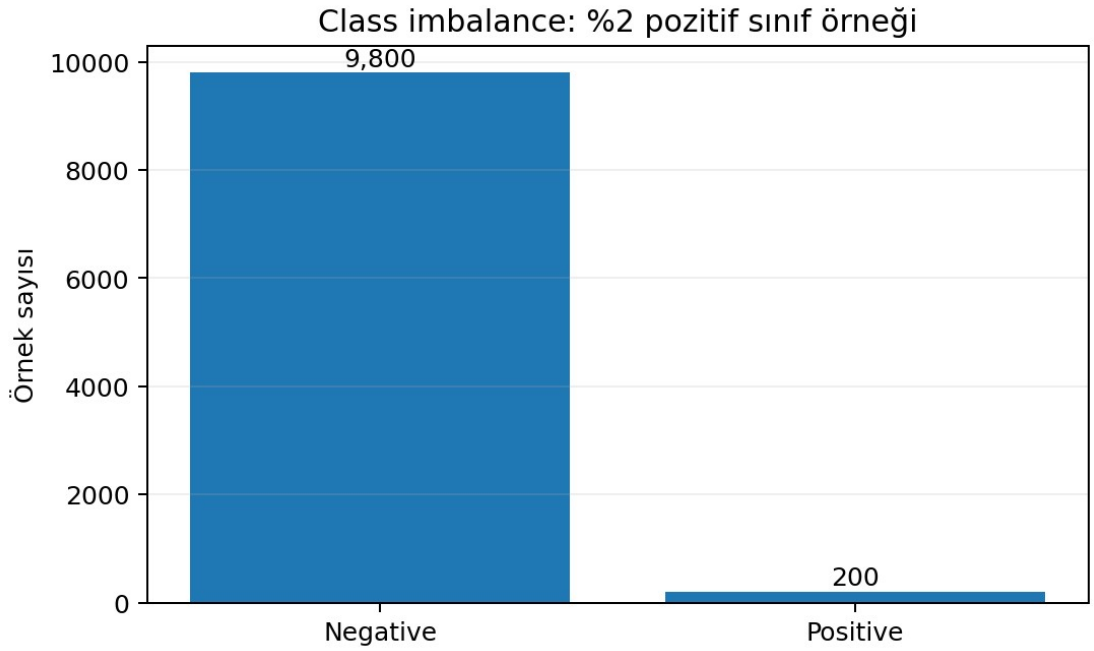
23. IMBALANCED DATA

Class Weights • Sampling • SMOTE • Threshold Tuning • Cost-Sensitive Learning



Bu modülün amacı
Dengesiz sınıf problemlerinde accuracy tuzağına düşmeden, doğru metric, validation, weighting, resampling ve threshold stratejilerini birlikte kullanarak gerçek iş maliyetini optimize etmeyi öğrenmek.

Kazanım	Modül sonunda
Problem tanıma	Class imbalance'ın ne zaman gerçek problem, ne zaman doğal prevalence olduğunu ayırt edebilirsin.
Metric seçimi	PR-AUC, precision, recall, F1, recall@k ve business cost kullanabilirsin.
Training stratejileri	Class weights, undersampling, oversampling ve SMOTE'yi doğru yerde kullanabilirsin.
Threshold	0.5 default'una bağlı kalmadan operating point seçebilirsin.
Leakage kontrolü	Sampling ve preprocessing adımlarını CV fold içinde tutabilirsin.
Business karar	Fraud/churn gibi problemlerde model skorunu kapasite ve maliyetle bağlayabilirsin.

1. Imbalanced data nedir?

Bir sınıf diğerinden çok daha az gözleniyorsa class imbalance vardır. Örneğin fraud oranı %0.5, churn oranı %8 veya kritik arıza oranı %0.2 olabilir.

Dengesizlik tek başına hata değildir
Pozitif sınıfın nadir olması gerçek dünyanın doğal özelliği olabilir. Ama evaluation ve training yaklaşımını buna göre değiştirmek gerekir.

2. Accuracy tuzağı

10,000 işlemin 9,900'ü normal ve 100'ü fraud ise herkese normal diyen model %99 accuracy alır ama tek bir fraud bile yakalayamaz.

Model	Accuracy	Recall (fraud)	İş değeri
Herkes normal	%99	%0	Yok
Risk modeli	%97	%75	Potansiyel olarak yüksek
Review odaklı model	%95	%90	Maliyete göre daha iyi olabilir

Ana soru
Model toplam doğruluğu mu optimize etmeli, yoksa nadir ve pahalı olayları yakalamayı mı?

3. Prevalence ve base rate

Positive prevalence = positive_count / total_count. Bu oran metric yorumunu doğrudan etkiler.

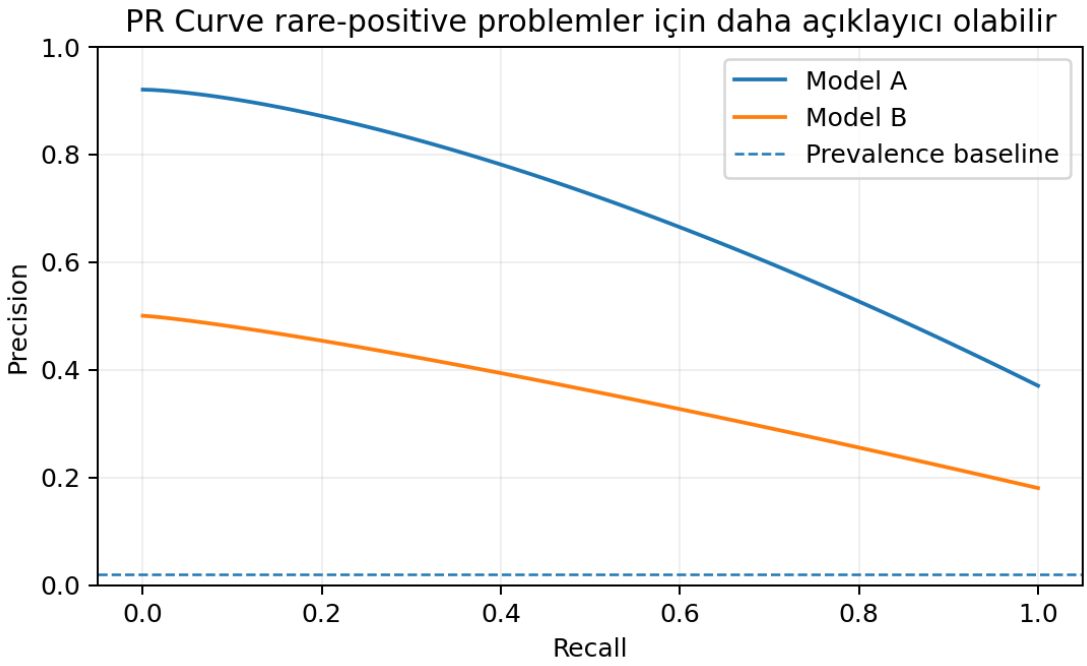
```
prevalence = y.mean()
print(prevalence)
```

PR baseline
Precision-Recall değerlendirmesinde rastgele modelin precision baseline'ı yaklaşık positive prevalence seviyesidir.

4. Doğru metric seçimi

Metric	Ne ölçer?	Rare positive için
Accuracy	Genel doğruluk	Genellikle zayıf
Precision	Pozitif tahminlerin temizliği	Önemli
Recall	Gerçek pozitifleri yakalama	Önemli
F1	Precision-recall dengesi	Tek operating point için
ROC-AUC	Ranking/discrimination	Kullanışlı ama iyimser olabilir
PR-AUC	Precision-recall trade-off	Çok uygun
Log-loss	Probability kalitesi	Calibration ile birlikte
Recall@k	Kapasite altında pozitif yakalama	Operasyonel olarak güçlü

5. PR Curve neden önemlidir?

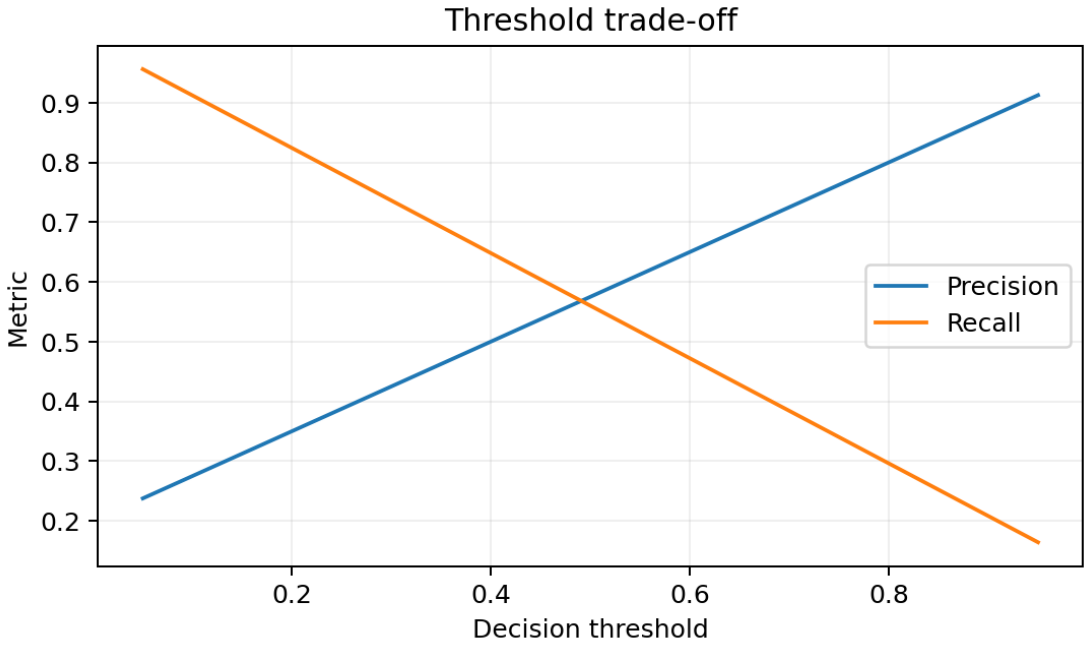


Rare-positive problemlerde PR curve, pozitif sınıf üzerindeki performansı daha doğrudan gösterir.

ROC-AUC yine değerli
ROC-AUC'yi tamamen bırakmak gerekmez; ama ağır dengesizlikte PR-AUC ve operating point metricleri daha açıklayıcı olabilir.

6. Precision - Recall trade-off

Threshold düşürüldüğünde daha fazla örnek pozitif seçilir. Recall genellikle artar, precision düşebilir.



7. Threshold tuning

```
proba = model.predict_proba(X_valid)[:, 1]

threshold = 0.25
pred = (proba >= threshold).astype(int)
```

0.5 kutsal değildir
Classification modelinin ürettiği probability ile business action threshold'u ayrı kavramlardır. Threshold validation üzerinde seçilir.

8. Business-cost threshold

False Negative ve False Positive maliyetlerini parasal veya utility cinsinden tanımlayabilirsen threshold seçimini doğrudan expected cost ile yapabilirsin.

```
total_cost = (
    fn_cost * false_negatives
    + fp_cost * false_positives
)
```

Senaryo	FN maliyeti	FP maliyeti	Muhtemel yön
Fraud	Çok yüksek	Review maliyeti	Recall lehine
Spam filtre	Önemli e-postayı kaçırmak yüksek	Spam görmek daha düşük	Precision lehine
Churn outreach	Kaçırılan churn fırsatı	Arama/indirim maliyeti	Kapasiteye bağlı

9. Class weights

Loss içinde minority class hatalarına daha fazla ağırlık verilir. Veri kopyalamadan decision boundary değişebilir.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(
    class_weight="balanced",
    max_iter=2000
)
```

balanced ne yapar?
Sınıf frekanslarına ters orantılı yaklaşık ağırlık üretir; fakat business cost'un doğru temsili olmak zorunda değildir.

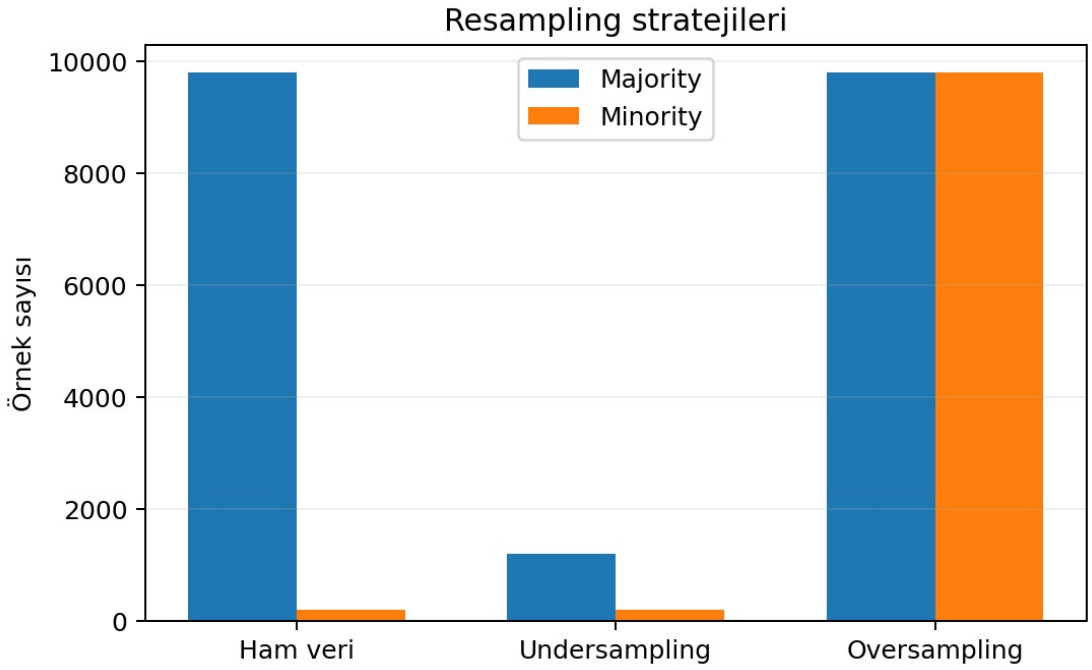
10. Manual class weights

```
model = LogisticRegression(
    class_weight={0: 1.0, 1: 6.0},
    max_iter=2000
)
```

Tuning
Class weight ratio da hyperparameter gibi CV/validation üzerinde optimize edilebilir.

11. Undersampling

Majority class örneklerinin bir kısmını training set'ten çıkarır.



Avantaj	Risk
Training hızlanır	Faydalı majority bilgisini kaybedebilir
Huge datasetlerde pratik	Decision boundary bozabilir
Basit baseline	Random örneklem şansa duyarlı

Ne zaman?
Majority sınıf milyonlarca çok benzer örnek içeriyorsa computational avantaj sağlayabilir.

12. Random Oversampling

Minority örneklerini tekrar ederek sınıf temsilini artırır.

Avantaj	Risk
Bilgi kaybetmez	Aynı örnekleri tekrar ederek overfit olabilir
Basit	Dataset büyür
Birçok modelle çalışır	Noise minority örnekler de çoğalır

13. SMOTE

SMOTE, minority örnekler arasında sentetik yeni noktalar üretir. Komşuluk yapısından interpolation yapar.

Kritik sezgi
SMOTE yeni gerçek gözlem yaratmaz; feature uzayında sentetik training örnekleri oluşturur.

```
from imblearn.over_sampling import SMOTE
```

```
smote = SMOTE(
    sampling_strategy=0.5,
    random_state=42
)

X_res, y_res = smote.fit_resample(
    X_train,
    y_train
)
```

14. SMOTE ne zaman riskli?

- High-dimensional sparse data.
- Categorical feature'lar ham haldeyken.
- Minority class içinde güçlü cluster farkları varsa.
- Outlier/noisy minority noktalar varsa.
- Zaman serisi veya ordered observations.

SMOTENC
Karma numeric + categorical veri için categorical-aware varyantlar gerekebilir.

15. Sampling leakage

SMOTE/oversampling split öncesi tüm veriye uygulanırsa validation örnekleri sentetik training örneklerinin oluşumuna katkı sağlayabilir.

Yanlış	Doğru
Önce SMOTE, sonra train/valid split	Önce split, SMOTE sadece training
CV dışı SMOTE	Her CV train fold içinde sampling
Validation'ı balance etmek	Validation doğal prevalence ile kalmalı

Altın kural
Sampling bir preprocessing adımıdır ve yalnız training fold üzerinde çalışmalıdır.

16. imbalanced-learn Pipeline

```
from imblearn.pipeline import Pipeline
from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ("scale", StandardScaler()),
    ("smote", SMOTE(random_state=42)),
    ("model", LogisticRegression(max_iter=2000))
])
```

Neden sklearn Pipeline değil?

Sampling transformer gibi yalnız X'i değil y'yi de değiştirir. imblearn Pipeline bu akışı doğru yönetir.

17. Cross-validation ile sampling

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

scores = cross_validate(
    pipe,
    X,
    y,
    cv=cv,
    scoring={
        "pr_auc": "average_precision",
        "roc_auc": "roc_auc",
        "recall": "recall"
    }
)
```

Validation prevalence
CV validation fold'ları doğal class dağılımını korur; resampling yalnız training fold'da yapılır.

18. Stratified split

Rare positive problemlerde split sonrası bazı fold'larda çok az pozitif kalmaması için class oranını korumak önemlidir.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

19. Group ve time imbalance

Stratification her problemi çözmez. Aynı kullanıcıdan tekrar kayıt varsa group split, zamana bağlı fraud/churn varsa temporal split gerekir.

Veri yapısı	Validation
IID + rare class	StratifiedKFold
Aynı entity tekrar ediyor	Group-aware split
Fraud/churn zaman içinde	Temporal split

Time + group birlikte	Deployment gerçekliğine özel split

Öncelik
Class oranını korumaktan önce leakage'i önle. Yanlış split ile mükemmel class balance işe yaramaz.

20. Tree-based modellerde imbalance

- Random Forest: class_weight kullanılabilir.
- XGBoost/LightGBM: positive class weight parametreleri kullanılabilir.
- CatBoost: class weights / auto class weights.

```
# XGBoost mantığı
scale_pos_weight = (
    n_negative / n_positive
)
```

Başlangıç değeri
Negatif/pozitif oranı iyi başlangıç olabilir; optimum değer validation business metricine göre değişebilir.

21. Balanced Random Forest

Bazı ensemble varyantları her ağacın training örneklemesinde minority/majority dengelemesi yapar.

Artı
Bagging içinde sampling entegre edilir. Yine de validation doğal prevalence üzerinde yapılmalıdır.

22. Focal Loss

Focal Loss kolay örneklerin katkısını azaltıp zor örneklere daha fazla ağırlık verir. Özellikle dense detection ve ağır imbalance senaryolarında kullanılır.

Her probleme şart değil
Tabular fraud/churn için class weights, boosting ve threshold tuning genellikle önce denenmesi gereken daha basit baseline'lardır.

23. Probability calibration etkisi

Class weights veya resampling model probability dağılımını değiştirebilir. Ranking iyi olsa bile probability'ler gerçek prevalence ile uyumsuz olabilir.

- Calibration curve kontrol et.
- Log-loss / Brier score kontrol et.
- Gerekirse calibration set veya cross-validated calibration kullan.

Risk skoru
Skoru gerçek probability olarak kullanacaksan calibration özellikle önemlidir.

24. Resampling ve calibration

Training prevalence yapay olarak değiştirildiğinde raw model probability'si production prevalence'ı doğrudan temsil etmeyebilir.

Pratik çözüm
Validation/test'i doğal prevalence ile bırak ve calibration'ı bu doğal dağılım üzerinde değerlendir.

25. Top-k evaluation

Fraud review veya churn outreach kapasitesi sınırlıysa threshold yerine en yüksek skorlu k kayıt seçilebilir.

Metric	İş anlamı
Precision@k	İncelenen k kaydın ne kadarı gerçek positive?
Recall@k	Tüm positives'ın ne kadarı ilk k içinde?
Lift@k	Top-k positive rate / genel prevalence

Operasyon kapasitesi
Günde 1,000 review yapabiliyorsan precision@1000/recall@1000 çoğu zaman F1'den daha değerli olabilir.

26. Cost-sensitive learning

Amaç yalnız sınıf dengesini düzeltmek değil, hata maliyetlerini loss veya karar mekanizmasına doğrudan katmaktır.

Yaklaşım	Nerede maliyet kullanılır?
Class weights	Training loss
Threshold tuning	Post-training decision
Custom objective	Model optimization
Expected utility	Final model/action seçimi

27. Fraud vakası

1,000,000 günlük işlem, fraud prevalence %0.3 ve manuel review kapasitesi 4,000 olsun.

Adım	Karar
Metric	PR-AUC + precision@4000 + recall@4000
Validation	Temporal holdout
Training	class weight / boosting / kontrollü sampling
Threshold	Review kapasitesine göre top-4000
Calibration	Risk score kullanılacaksa kontrol

Business	Fraud loss prevented - review cost

Accuracy burada gereksiz
%99+ accuracy fraud yakalamayan modelde bile mümkün olabilir.

28. Churn vakası

Churn prevalence %7 ve retention ekibi haftada yalnız 15,000 müşteriye ulaşabiliyor.

Yanlış yaklaşım	Daha iyi yaklaşım
0.5 threshold	Top-15k risk score
Accuracy	Precision@15k, Recall@15k
Her müşteriye teklif	Expected retention value
Random split	Prediction zamanı varsa temporal split

29. Medical screening vakası

Rare disease screening'de FN maliyeti çok yüksek olabilir; ancak aşırı düşük threshold çok fazla unnecessary follow-up yaratabilir.

Amaç
Yüksek recall + kabul edilebilir precision/specificity. Klinik kullanımda metric seçimi uzman ve güvenlik gereksinimleriyle birlikte yapılır.

30. Sampling stratejisi seçimi

Durum	İlk denenecek
Moderate imbalance	Class weights + threshold
Huge majority dataset	Undersampling + weighting
Small minority sample	Weights + careful oversampling
Tabular smooth feature space	SMOTE denenebilir
Sparse text	Class weights / linear model; SMOTE dikkat
Time series	Temporal weighting; SMOTE genelde dikkat

31. Model karşılaştırma matrisi

Yaklaşım	Artı	Eksi
Class weights	Basit, dataset büyümez	Probability etkilenebilir
Undersampling	Hızlı	Bilgi kaybı
Oversampling	Bilgi kaybı yok	Overfit/data büyümesi
SMOTE	Yeni sentetik minority	Noise/boundary riski
Threshold tuning	Business'e doğrudan	Training representation değişmez
Cost-sensitive objective	Maliyeti optimize eder	Daha özel uygulama

32. Hyperparameter tuning

Sampling oranı, class weight, threshold ve model hyperparameter'ları birlikte trade-off oluşturur.

```
param_grid = {
    "smote__sampling_strategy": [0.2, 0.5, 1.0],
    "model__C": [0.1, 1, 10]
}
```

Threshold ayrı olabilir
GridSearchCV model pipeline'ını PR-AUC gibi threshold-free metric ile seçip threshold'u sonrasında validation'da business cost'a göre ayarlamak temiz bir yaklaşımdır.

33. Common anti-patterns

- Tüm dataset'e SMOTE uygulayıp sonra split yapmak.
- Validation/test set'i balance etmek.
- Accuracy ile model seçmek.
- ROC-AUC yüksek diye rare-positive performansını yeterli sanmak.
- Class weight ve threshold etkisini birbirine karıştırmak.
- Test set'te threshold tuning yapmak.
- Resampling sonrası probability'leri calibrated varsaymak.
- Entity/time leakage'i sırf stratification için görmezden gelmek.
- Synthetic örnekleri gerçek veri gibi yorumlamak.
- Business capacity ve error cost'u evaluation'a katmamak.

34. İyi workflow

1. Prevalence ve business hata maliyetini ölç.
2. Deployment-gerçekçi split kur.
3. Natural validation/test distribution'ını koru.
4. Baseline ve class-weighted baseline oluştur.
5. PR-AUC + operating-point metriclerini kullan.
6. Sampling'i yalnız training fold içinde dene.
7. Threshold'u validation business cost/capacity ile seç.
8. Calibration ve segment error analizi yap.
9. Test set'te tek final evaluation yap.
10. Production prevalence/drift değişimini izle.

35. Python - evaluation şablonu

```
from sklearn.metrics import (
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    average_precision_score
)

proba = model.predict_proba(X_valid)[:, 1]
threshold = 0.30
pred = (proba >= threshold).astype(int)

print("precision:", precision_score(y_valid, pred))
print("recall:", recall_score(y_valid, pred))
print("f1:", f1_score(y_valid, pred))
print("roc_auc:", roc_auc_score(y_valid, proba))
print("pr_auc:", average_precision_score(y_valid, proba))
```

36. Mülakat soruları

Soru	Kısa cevap
Imbalanced data nedir?	Bir sınıfın diğerlerine göre çok daha az temsil edilmesidir.
Accuracy neden kötü olabilir?	Majority sınıfı tahmin ederek yüksek accuracy alınabilir.
Rare positive için hangi metric?	PR-AUC, precision, recall ve operating point metricleri.
Class weight ne yapar?	Training loss'ta bazı sınıf hatalarını daha pahalı yapar.
Oversampling riski?	Overfitting ve dataset büyümesi.
Undersampling riski?	Majority bilgisini kaybetmek.
SMOTE nedir?	Minority komşuları arasında sentetik örnek üretir.
SMOTE nerede uygulanmalı?	Sadece training fold içinde.
Validation set balance edilmeli mi?	Hayır, production prevalence'ı temsil etmelidir.
Threshold neden 0.5 değil?	Business hata maliyetleri ve kapasite farklıdır.
PR-AUC neden?	Rare-positive performansını daha doğrudan gösterir.
Class weight calibration'ı etkiler mi?	Evet, probability dağılımını etkileyebilir.
scale_pos_weight başlangıç değeri?	Kabaca n_negative / n_positive.
Top-k ne zaman?	Operasyon kapasitesi sabit olduğunda.
Focal loss ne yapar?	Kolay örnekleri down-weight edip zor örneklere odaklanır.

37. Uygulama alıştırmaları

11. Synthetic %1 positive dataset oluştur; accuracy ve PR-AUC farkını göster.
12. Logistic Regression için class_weight None vs balanced karşılaştır.
13. RandomUnderSampler, RandomOverSampler ve SMOTE sonuçlarını karşılaştır.
14. Sampling'i split öncesi uygulayıp leakage etkisini kontrollü deneyle gözlemle.
15. Threshold 0.05-0.95 aralığında precision/recall/F1 eğrisi çiz.
16. FN cost=100, FP cost=5 için minimum-cost threshold bul.
17. PR-AUC ile ROC-AUC model ranking'lerini karşılaştır.
18. Resampling sonrası calibration curve kontrol et.
19. Precision@k ve Recall@k fonksiyonu yaz.

20. Temporal fraud dataset için class-weighted boosting modeli tasarla.

38. Mini portföy projesi

Proje: Fraud Detection Under Extreme Class Imbalance

Çıktı	İçerik
01_problem.md	Prevalence, review capacity, FP/FN cost
02_baseline.ipynb	Dummy + weighted Logistic
03_sampling.ipynb	Under/over/SMOTE leakage-safe benchmark
04_boosting.ipynb	Weighted XGBoost/LightGBM
05_threshold.ipynb	PR curve, top-k, expected cost
06_calibration.ipynb	Probability reliability
src/pipeline.py	imblearn Pipeline
report.md	Final operating point ve business impact

CV cümlesi
Built an extreme-imbalance fraud detection pipeline comparing class weighting, resampling and SMOTE with leakage-safe cross-validation, PR-AUC evaluation, calibration and cost-based threshold optimization.

39. Final cheat sheet

Kavram	Hatırlanacak fikir
Prevalence	Positive sınıf oranı
Accuracy	Rare positive'da yanıltıcı olabilir
PR-AUC	Rare-positive ranking özeti
Class weight	Loss'ta minority hatasını ağırlaştır
Undersampling	Majority azalt
Oversampling	Minority çoğalt
SMOTE	Sentetik minority örnekleri
Threshold	Business operating point
Top-k	Kapasite tabanlı aksiyon
Calibration	Probability güvenilirliği
Stratification	Class oranını koru
Leakage-safe sampling	Sadece train fold
Cost-sensitive	Hata maliyetini optimize et
Focal loss	Zor örneklere odaklan
Natural validation	Production prevalence'ı koru

Bu modülden sonra
Explainable AI bölümünde özellikle rare-positive modellerde hangi feature'ların alarm ürettiğini SHAP ve permutation importance ile inceleyebileceksin.